

سوال: چرا از Kafka به جای دیتابیس رابطه‌ای برای لایه دریافت سفارش استفاده شده است؟

پاسخ:

۱. عملکرد (Performance):

Kafka داده‌ها را به صورت پشت سر هم (sequential) روی دیسک می‌نویسد و سر بار اضافی ایندکس و جستجوی تصادفی (random I/O) را ندارد. این ویژگی باعث می‌شود توان عملیاتی (throughput) بسیار بالاتری نسبت به دیتابیس‌های رابطه‌ای برای جریان داده‌های لحظه‌ای داشته باشد.

۲. مقیاس‌پذیری (Scalability):

Kafka به راحتی با اضافه کردن بروکرهای جدید به صورت افقی (horizontal) مقیاس می‌شود. در دیتابیس‌های رابطه‌ای، مقیاس افقی کار دشواری است و نیاز به تکنیک‌هایی مثل شاردینگ (sharding) دارد.

۳. الگوی دسترسی (Access pattern):

Kafka بر پایه مدل انتشار/اشتراک (publish/subscribe) طراحی شده است و برای جابه‌جایی پیام بین تولیدکننده و مصرف‌کننده بهینه می‌باشد. دیتابیس‌های رابطه‌ای برای پرس‌وجوهای نقطه‌ای و گزارش‌گیری تحلیلی مناسب‌تر هستند.

۴. مناسب برای پردازش لحظه‌ای (real-time streaming):

Kafka ذاتاً از جریان پیوسته داده (data stream) پشتیبانی می‌کند و با ابزارهایی مثل اسپارک استریمینگ (Spark Streaming) هماهنگی کاملی دارد. دیتابیس‌های رابطه‌ای چنین قابلیتی ندارند.

یک سناریو که دیتابیس رابطه‌ای بهتر است:

سیستم‌های بانکی یا حسابداری که نیاز به تراکنش‌های اسید (ACID) و یکپارچگی ارجاعی (referential integrity) دارند و نرخ ورود داده پایین است. مثلاً ذخیره کردن فاکتورهای نهایی.

پاسخ سوال دوم (II)

سوال: تفاوت پردازش stateless و stateful چیست؟ چرا در لایه ingestion پردازش stateless ترجیح داده می‌شود و چه ریسک‌هایی در صورت وابسته شدن کانسیومر به state وجود دارد؟

پاسخ:

پردازش بدون وضعیت (stateless processing): هر پیام کاملاً مستقل از پیام‌های قبلی یا بعدی پردازش می‌شود. کانسیومر هیچ اطلاعاتی از رویدادهای گذشته نگه نمی‌دارد و به دیتابیس یا حافظه خارجی دسترسی ندارد. اعتبارسنجی‌هایی که ما در بخش ۲ انجام دادیم (شماره تلفن، تداخل حالت، صحت قیمت) stateless هستند.

پردازش با وضعیت (stateful processing): کانسیومر باید اطلاعاتی از رویدادهای قبلی (مثل شمارش تعداد سفارش‌های یک کاربر در بازه زمانی) یا وضعیت ذخیره شده در یک دیتابیس خارجی را نگه دارد.

چرا stateless در لایه ingestion ترجیح داده می‌شود؟

سادگی و سرعت: نیازی به خواندن/نوشتن از دیتابیس یا حافظه مشترک نیست، بنابراین تأخیر (latency) پایین و توان (throughput) بالا است.

مقیاس‌پذیری آسان: می‌توانیم چندین نمونه (instance) از کانسیومر را بدون هیچ هماهنگی اضافه کنیم.

تحمل خطا (fault tolerance): اگر کانسیومر crash کند، پس از راه‌اندازی مجدد، از روی آخرین offset به خواندن ادامه می‌دهد و نیازی به بازسازی state نیست.

ریسک‌های stateful شدن کانسیومر در لایه ingestion:

افزایش تأخیر و پیچیدگی به دلیل دسترسی به حافظه مشترک یا دیتابیس.

در صورت بروز rebalance یا crash، ممکن است state از دست برود یا دوباره‌سازی آن زمان‌بر باشد.

مقیاس‌افقی مشکل می‌شود؛ باید پارتیشن‌بندی state را مدیریت کرد.

نقض شرط صورت پروژه که گفته بود "No database access is allowed at this stage".

پاسخ سوال سوم (III)

سوال: چرا Kafka را به عنوان "distributed commit log" توصیف می‌کنند؟ این طراحی چه مزایایی برای replay، مصرف‌کننده‌های مستقل و تحمل خطا دارد؟

پاسخ:

معنی commit log در عمل:

Kafka داده‌ها را به صورت ترتیبی (sequential) و فقط در انتها (append-only) روی دیسک ذخیره می‌کند. هر پیام یک آفست (offset) دارد و پس از خوانده شدن توسط مصرف‌کننده حذف نمی‌شود. این رفتار برخلاف صف‌های سنتی (مثل RabbitMQ) است که پیام را پس از مصرف حذف می‌کنند. به همین دلیل می‌گویند Kafka یک "دفتر ثبت توزیع‌شده" است.

مزایا:

بازپخش پیام (Message replay):

هر مصرف‌کننده می‌تواند با تنظیم offset خود، پیام‌ها را از ابتدا (یا از هر نقطه دلخواه) دوباره بخواند. در کد ما این کار با `'auto_offset_reset'='earliest'` انجام شده است.

مصرف‌کننده‌های مستقل (Multiple independent consumers):

هر مصرف‌کننده (یا گروه مصرف‌کننده) آفست مخصوص خود را نگه می‌دارد. بنابراین دو مصرف‌کننده با `group.id` متفاوت می‌توانند به طور کامل مستقل از هم کل پیام‌های یک تاپیک را بخوانند بدون اینکه روی یکدیگر تأثیر بگذارند.

تحمل خطا (Fault tolerance):

هر پارتیشن چندین رپلیکا (replica) دارد که روی بروکرهای مختلف ذخیره می‌شوند. اگر یک بروکر از کار بیفتد، مصرف‌کننده به طور خودکار به رپلیکای سالم دیگر سوئیچ می‌کند. همچنین ذخیره‌سازی دائمی (persistent) روی دیسک باعث می‌شود داده‌ها از بین نروند.